Assignment5 – Markov Decision Procedures and Decision Trees

Given: May 20 Due: May 25

Problem 5.1 (HMMs in Python)

Implement filtering, *prediction* and *smoothing* for *HMMs* in Python by completing the *implementation* of `hmm.py` at `https://kwarc.info/teaching/AI/resources/AI2/hmm/`.

Hint: This problem uses `numpy`, which is a Python *library* for working with arrays/matrices. If you have never worked with `numpy` before, you can find many high-quality introductions online. Due to its popularity and frequent use for machine learning etc., it is definitely worth getting to know `numpy`. That being said, you only need very few and basic numpy functions for this assignment, which you should be able to find without problems (e.g. searching for *numpy matrix multiplication*).

Problem 5.2 (Value Iteration for Navigation)

Implement value iteration for an agent navigating worlds like the 4x3 world from the *lecture notes*. The agent has four possible actions: *right*, *up*, *left*, *down*. The probability of actually moving in the intended direction is p and the probability of moving in one of the orthogonal directions is $\frac{1-p}{2}$ respectively. For example, if $p = 0.8$ and the chosen action is *up*, the agent will actually move up with a probability of $p = 0.8$ and will move left and right with a probability of 0.1 each. If the agent ends up moving in a direction that has no free adjacent square, it will remain on its current square instead. For example, if the agent is on square $(0, 0)$ with the action *up*, it will end up on square $(0, 1)$ with a probability of p, on square $(1, 0)$ with a probability of $\frac{1-p}{2}$ and on square $(0, 0)$ with a probability of $\frac{1-p}{2}$.

(0, 2) -0.040 → 0.647	(1, 2) -0.040 → 0.753	(2, 2) -0.040 → 0.855	(3, 2) 1.000 T 1.000
(0, 1) -0.040 ↑ 0.557	(1, 1) W	(2, 1) -0.040 ↑ 0.569	(3, 1) -1.000 T -1.000
(0, 0) -0.040 ↑ 0.465	(1, 0) -0.040 ← 0.386	(2, 0) -0.040 ↑ 0.451	(3, 0) -0.040 ← 0.230

Results for 4x3 world with $p = 0.8$, $\gamma = 0.95$, $\varepsilon = 0.001$.

A skeleton *implementation* with technical instructions can be found at `https://kwarc.info/teaching/AI/resources/AI2/mdp/`. It also allows the visualization of the computed utilities and policy (see figure above): Each square is annotated with the coordinates, the reward, the computed policy and the computed utility. Walls and terminal nodes don't have a policy and are marked with W and T respectively.

Hint: You will also have to compute a policy based on the utilities obtained from value iteration. For that, you should pick the actions that *maximize* the expected utility. A common mistake is the assumption that the best policy is always to go in the direction of the square with the maximal utility.

Problem 5.3 (POMDPs)

An agent is moving on an infinite discrete line. We represent its position as an integer.

Initially, it is in position 4 with 60% probability or 5 with 40% probability. Its goal is to reach position 0.

Its actions are to move left or right 1 step or to do nothing. Making a step succeeds 75% of the time; the remaining times, the agent does not move. Doing nothing always succeeds, i.e., does not change the state.

After each action the agent measures which state it is in. This measurement is correct 50% of the time and one off in either direction 25% of the time.

We want to model this as POMDP.

Hint: We often have to give probability distributions, e.g., over S. Such a distribution is a function $p : S \to [0;1]$. Often $p(s') = 0$ for all but finitely many values, e.g., $p(s_0) = p(s_1) = 0.5$ and $p(s) = 0$ otherwise. Such a function p is a set of pairs $(s', r) \in S \times [0;1]$ where $r = p(s')$. In that case, we can write p efficiently by listing only the non-zero values, e.g., $p = \{(s_0, 0.5), (s_1, 0.5)\}$. Technically, we have $p = \{(s_0, 0.5), (s_1, 0.5)\} \cup \{(s, 0) : s \in S \setminus \{s_0, s_1\}\}$, but we can clarify that with a comment that all omitted values are 0. That is a convenient way to write a concrete distribution over a large domain where only a few values are non-zero.

---

1. Give the state space S and the set A of actions.
2. Give the transition model $T(s, a)$ as a probability distribution over states.
3. Give the initial state I and define an appropriate reward function R.
4. Give the domain E of observations and the observation model $O(s, e)$.
5. The agent initially moves 1 step to the right in belief state $b_0 = I$. What are the next belief state b_1 (before taking the next measurement) and its expected reward $\rho(b_1)$ and the distribution of the next observation E_1?

6. After the move from the previous question, we measure that we are in state 4. What is the updated belief state?
7. Give the corresponding MDP $\langle S', A', I', T', R' \rangle$. For T', don't give the precise definition and simply explain how to create it.

Problem 5.4 (Sunbathing)

Eight people go sunbathing. They are categorized by the attributes Hair and Lotion and the result of whether they got sunburned.

Name	Hair	Lotion	Result: Sunburned
Sarah	Light	No	Yes
Dana	Light	Yes	No
Alex	Dark	Yes	No
Annie	Light	No	Yes
Julie	Light	No	No
Pete	Dark	No	No
John	Dark	No	No
Ruth	Light	No	No

1. Which quantity does the information theoretic decision tree learning algorithm use to pick the attribute to split on?
2. Compute that quantity for the attributes Hair and Lotion. (Simplify as much as you can without computing logarithms.)
3. Assuming the logarithms are computed, how does the algorithm pick the attribute?

Problem 5.5 (Decision Trees)

You observe the values below for 6 different football games of your favorite team. You want to construct a decision tree that predicts the result.

#	Day	Weather	Location	Opponent	Result
1	Monday	Rainy	Home	Weak	Win
2	Monday	Sunny	Home	Weak	Win
3	Friday	Rainy	Away	Strong	Loss
4	Sunday	Sunny	Home	Weak	Win
5	Friday	Cloudy	Home	Strong	Draw
6	Sunday	Sunny	Home	Strong	Draw

1. Assume you choose attributes in the order
   $Opponent, Location, Weather, Day$.
   Give the resulting decision tree.
2. Without using the above observations, give the formula for the information gain of the attribute $Opponent$.
3. Using the above observations, give the results of
   - $I(P(Result)) =$

- $P(Result = Loss \mid Opponent = Strong) =$

You do not have to compute irrational logarithms.